Compresor Huffman

1. Descripción General

El proyecto consiste en desarrollar un sistema de compresión de texto basado en el **algoritmo de Huffman**, el cual asigna códigos binarios de longitud variable a los caracteres según su frecuencia:

- Caracteres **más frecuentes** → códigos **más cortos**
- Caracteres **menos frecuentes** → códigos **más largos**

El algoritmo construye un **árbol binario** (Árbol de Huffman) utilizando una **cola de prioridad** (min-heap) para combinar los dos nodos de menor frecuencia repetidamente hasta formar un solo árbol.

El sistema debe permitir:

1. Analizar frecuencias
2. Construir el árbol de Huffman
3. Generar la tabla de códigos
4. Codificar (comprimir) un texto
5. Decodificar (descomprimir) para verificar integridad
6. Reportar métricas de compresión y eficiencia

2. Objetivo del Proyecto

Desarrollar un sistema que:

- Permita ingresar un texto (o cargarlo desde consola).
- Calcule frecuencias de caracteres.
- Construya el árbol de Huffman.
- Genere códigos binarios prefijo-libres.
- Comprima el texto y muestre el resultado (o una representación).
- Descomprima el texto y valide que coincida con el original.
- Compare al menos dos enfoques o variantes cuando sea aplicable.

- Analice complejidad temporal y espacial, y reporte métricas reales.

3. Requerimientos Funcionales

El sistema debe permitir:

3.1 Entrada

- Ingresar una cadena de texto (mínimo 20 caracteres para pruebas).
- (Opcional) Permitir espacios y signos de puntuación.
- (Opcional) Permitir modo “archivo” si usted lo permite; si no, solo consola.

3.2 Construcción de Huffman

- Calcular la frecuencia de cada símbolo.
- Construir el árbol de Huffman.
- Generar tabla de códigos por carácter.

3.3 Compresión

- Codificar el texto original en bits (string de 0/1 o estructura bitset).
- Mostrar:
    - tamaño original (en bits o bytes)
    - tamaño comprimido (en bits)
    - porcentaje de compresión

3.4 Descompresión

- Decodificar el texto comprimido.
- Verificar coincidencia exacta con el original:
    - Decodificación correcta
    - Error si no coincide.

3.5 Reportes

- Mostrar tabla de frecuencias.
- Mostrar tabla de códigos Huffman.

- Mostrar métricas comparativas.

4. Requerimientos Técnicos Obligatorios

4.1 Estructuras de Datos

Debe utilizar al menos:

Una estructura lineal

- Lista de caracteres/códigos para imprimir reportes
- Pila (opcional) para recorridos del árbol
- Lista para almacenar el texto comprimido/decodificado

Una estructura no lineal

- **Árbol binario** (Árbol de Huffman) (obligatorio)
- **Cola de prioridad** (min-heap) para construir el árbol (obligatorio)
- **Tabla hash / mapa** para frecuencias y códigos (obligatorio)

No se permite construir el árbol "a mano" o con ordenamientos repetidos sin cola de prioridad, salvo que se justifique formalmente el impacto en eficiencia (no recomendado).

4.2 Algoritmos (obligatorio comparar variantes)

Debe implementarse:

Variante A – Huffman estándar

- Calcular frecuencias
- Construir árbol usando min-heap
- Generar códigos recorriendo el árbol (DFS)

Variante B – Comparación de variante (elegir una)

Se debe comparar Huffman contra **alguna variante o escenario**, por ejemplo:

- **Comparación por distribución**:
    - Texto con caracteres muy repetidos vs texto uniforme
    - Comparar % de compresión
- **Comparación con codificación fija**:

  - ASCII fijo de 8 bits por caracter vs Huffman
- (Opcional) Huffman con "tie-break" determinista para reproducibilidad

Lo importante es que exista comparación cuantitativa: no solo "Huffman funciona", sino "cuándo mejora y cuánto".

4.3 Documentación (README)

El README.md debe incluir:

- Complejidad temporal y espacial de:
  - Conteo de frecuencias: $O(n)$
  - Construcción del árbol: $O(k\log\ k)$donde $k$es el número de símbolos distintos
  - Codificación/decodificación: $O(n)$en longitud del texto (con discusión)
- Métricas reales:
  - Tamaño original vs comprimido
  - % de compresión
  - Tiempo de compresión y descompresión
- Comparación entre escenarios/variantes:
  - Texto repetitivo vs uniforme (o fijo 8-bit vs Huffman)
- Discusión de escalabilidad:
  - Qué ocurre si aumenta el texto o los símbolos distintos.

5. Ejemplo de Entrada

Texto: ABRACADABRA

6. Ejemplo de Salida Esperada

```
Frecuencias:
A: 5
B: 2
R: 2
C: 1
D: 1
```

```
Códigos Huffman (pueden variar):
A: 0
B: 10
R: 110
C: 1110
D: 1111

Texto comprimido:
0 10 110 0 1110 0 1111 0 10 110 0
(Representación en bits concatenados)

Descompresión:
ABRACADABRA
Decodificación correcta

Métricas:
Tamaño original: 11 caracteres = 88 bits (ASCII 8-bit)
Tamaño comprimido: 23 bits
Compresión: 73.86%

Comparación:
Texto uniforme (ej. ABCDEFGHIJ):
Compresión: ~0% (o menor mejora)
```

(Los códigos y bits exactos pueden variar, pero deben ser prefijo-libres y la decodificación debe coincidir con el texto original.)

7. Validaciones Obligatorias

- Manejar texto vacío (reportar error).
- Manejar texto con un solo símbolo (caso especial de Huffman).
- Validar que la tabla de códigos sea prefijo-libre (por construcción).
- Validar que la descompresión reproduzca exactamente el texto original.

8. Interfaz (Consola)

Debe incluir un menú que permita:

1. Ingresar texto
2. Calcular frecuencias

3. Construir árbol Huffman
4. Mostrar códigos
5. Comprimir texto
6. Descomprimir texto
7. Mostrar métricas y comparación
8. Salir

9. Aspectos de Evaluación Clave

Se evaluará especialmente:

- Construcción correcta del árbol (min-heap).
- Generación correcta de códigos (recorrido del árbol).
- Compresión y descompresión correctas.
- Uso adecuado de hash/map para frecuencias y códigos.
- Comparación cuantitativa real.
- Análisis de complejidad y métricas.

10. Consideraciones en la Defensa

Durante la defensa, los estudiantes deben poder explicar:

- Por qué Huffman produce códigos prefijo-libres.
- Por qué usar min-heap reduce el costo de construcción.
- Diferencia entre codificación fija vs variable.
- Cuándo Huffman comprime mejor (distribución sesgada de frecuencias).
- Complejidad de construir el árbol y de codificar/decodificar.